

Aegis-AI — Complete Project Summary

1. Project Title

Aegis-AI: An Adaptive Multi-Agent Framework for Continuous Business Process Compliance Monitoring, Testing and Policy-Change Impact Analysis

2. Core Idea

Aegis-AI is an AI-based business-process compliance platform that continuously checks whether organizational activities follow internal policies and regulatory requirements.

Instead of depending only on fixed rules and manual audits, Aegis-AI combines:

- **Large Language Models (LLMs)**
- **Retrieval-Augmented Generation (RAG)**
- **Multi-agent AI**
- **Deterministic rule-based checking**
- **Policy change analysis**
- **Automatic compliance test generation**
- **Risk assessment**
- **Controlled remediation**
- **Independent verification**
- **Audit and evidence tracking**

The main goal is to make compliance **continuous, adaptive, explainable, and traceable**.

3. Problem Addressed

Organizations perform processes such as:

- Procurement
- Expense management
- Vendor onboarding
- Finance
- Employee operations
- Travel and reimbursement

These processes must follow policies and regulations.

Traditional compliance systems often depend on:

Predefined rules + manual verification + periodic audits

This creates problems when:

- Policies are frequently changed.
- Policies exist as PDF/DOCX documents.
- Transaction volumes become large.
- Existing rules need to be manually updated.
- Different systems monitor different compliance activities.
- AI-only decisions may be inconsistent or difficult to justify.
- Compliance decisions need reliable evidence.

Aegis-AI addresses these limitations by combining **AI reasoning with deterministic compliance checks and evidence retrieval**.

4. Overall Working

The basic concept can be represented as:

Policy Documents



Policy Understanding



Structured Compliance Controls



RAG / Knowledge Base



Continuous Monitoring



Rule-Based Compliance Check



Violation Detected



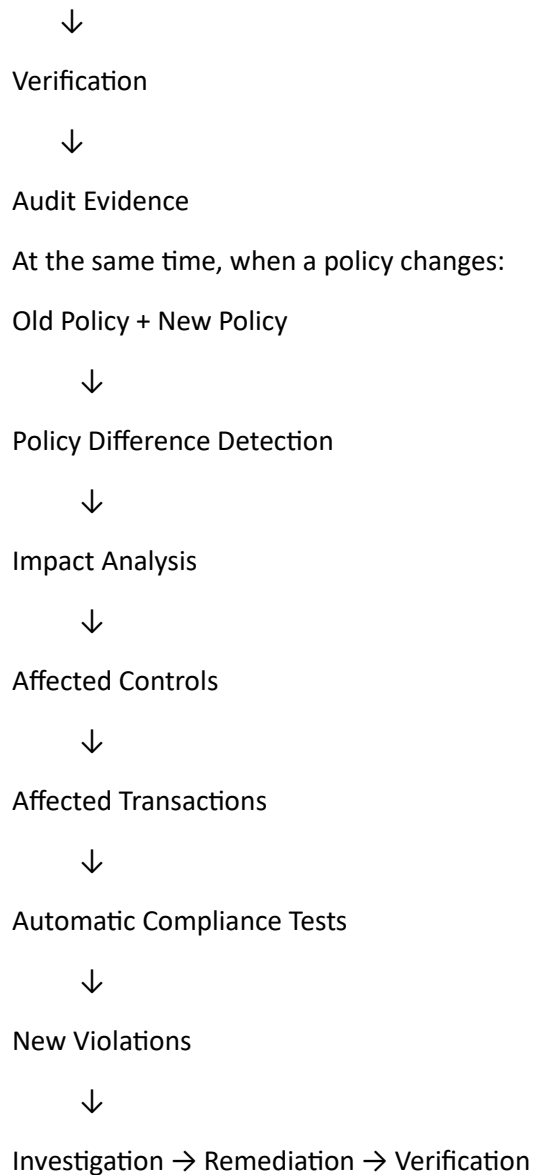
Investigation



Risk Assessment



Remediation



This policy-change loop is one of the important research aspects of Aegis-AI.

5. Policy Processing

Authorized users such as an **Admin or Compliance/Policy Officer** can upload policy documents.

Supported formats:

- PDF
- DOCX
- TXT

Example:

Procurement Policy v3.2

Finance approval required for purchases above ₹3,00,000

The system extracts the policy content and processes it.

Policy understanding

The LLM can convert natural-language requirements into structured compliance controls.

For example:

```
{  
  "control": "Finance Approval",  
  "condition": "purchase_amount > 300000",  
  "threshold": 300000,  
  "unit": "INR",  
  "required_role": "Finance",  
  "action": "approval_required"  
}
```

These structured controls can then be stored in **MySQL** and used by the compliance engine.

6. RAG — Retrieval-Augmented Generation

Aegis-AI uses **RAG** so that AI agents do not have to depend only on the knowledge stored inside an LLM.

The policy is:

Policy Document



Text Extraction



Chunking



Embedding Generation



ChromaDB

When an agent needs policy information:

Question / Transaction



Semantic Search



Relevant Policy Chunks



LLM



Evidence-based Response

This allows the agent to provide decisions based on the organization's actual policy.

7. Sentence Transformers

For semantic search, Aegis-AI uses:

Sentence Transformers — all-MiniLM-L6-v2

It converts policy text into numerical **embedding vectors**.

For example:

"Finance approval is required above ₹3 lakh"



Sentence Transformer



Embedding Vector



ChromaDB

When a relevant transaction or question is received, semantic similarity is used to retrieve related policy sections.

Why all-MiniLM-L6-v2?

It is:

- Lightweight
 - Fast
 - Suitable for semantic similarity
 - Suitable for document retrieval
 - Practical for a student project/deployment
-

8. ChromaDB

ChromaDB is used as the vector database.

It stores:

- Policy chunks
- Embeddings
- Policy metadata
- Version information
- Section information

Example:

Aegis Policy Knowledge Base

```
|
|└─ Procurement v3.1
|└─ Procurement v3.2
|└─ Expense v2.0
|└─ Vendor v4.0
└─ Travel v2.5
```

The original PDF/DOCX should be stored separately, while ChromaDB stores the semantic representation needed for retrieval.

9. MySQL

MySQL stores structured application and compliance data.

Important tables can include:

users

business_processes

process_steps

transactions

approvals

policies

policy_versions

policy_sections

compliance_controls

compliance_cases

risk_assessments
remediation_actions
verification_results
agent_runs
agent_actions
policy_changes
policy_impacts
evidence
audit_logs

MySQL therefore handles the structured operational side of the system, while **ChromaDB handles semantic retrieval**.

10. Deterministic Rule Engine

This is a very important part of Aegis-AI.

The system should **not send every transaction to an LLM**.

Exact compliance conditions are handled by a deterministic rule engine.

For example:

IF purchase_amount > ₹3,00,000

AND finance_approval = false

THEN

NON_COMPLIANT

The rule engine is useful for:

- Threshold checking
- Boolean conditions
- Required approvals
- Mandatory documents
- Transaction calculations
- Exact policy controls

This improves consistency and reduces the risk of unpredictable LLM decisions.

11. LLM

Aegis-AI uses an LLM for tasks that require language understanding and reasoning.

The LLM can perform:

- Policy interpretation
- Policy-to-control extraction
- Investigation
- Explanation generation
- Evidence interpretation
- Remediation planning
- Policy-change interpretation
- Ambiguity detection

A possible inference provider is **Groq**, using a suitable hosted LLM.

Important distinction:

Groq is the inference platform/provider; the actual model can be a model such as Llama.

12. Multi-Agent AI

Aegis-AI uses **LangGraph** to coordinate specialized agents.

Investigation Agent

Responsible for:

- Understanding the violation
 - Retrieving relevant policy
 - Examining transaction context
 - Collecting evidence
 - Explaining why the transaction is non-compliant
-

Risk Assessment

The system evaluates the severity of the violation.

Possible levels:

Low

Medium

High

Critical

Risk can consider factors such as:

- Transaction value
 - Policy severity
 - Type of violation
 - Business impact
 - Frequency
 - Previous violations
-

Remediation Agent

The Remediation Agent determines the appropriate corrective action.

Examples:

- Request missing document
- Request approval
- Request transaction correction
- Block transaction
- Escalate to compliance officer

High-impact actions should require **human approval**.

Verification Agent

The Verification Agent independently checks whether the remediation actually restored compliance.

For example:

Violation:

₹4,50,000 purchase without Finance approval



Remediation:

Finance approval requested



Verification:

Finance approval received?



YES → Compliance restored

This prevents the system from closing a case simply because an action was proposed.

13. LangGraph

LangGraph is used for multi-agent orchestration.

It manages the workflow between agents.

Conceptually:

Investigation Agent



Risk Assessment



Remediation Agent



Verification Agent

The workflow can also branch depending on risk.

For example:

Low Risk



Automatic remediation

Medium Risk



Human approval

High/Critical Risk



Mandatory human approval

14. Policy Change Analysis

One of the major features of Aegis-AI is its ability to handle **policy changes**.

Suppose:

Policy v3.1

Finance approval required above ₹5,00,000

Policy v3.2

Finance approval required above ₹3,00,000

Aegis-AI identifies that the threshold has changed.

The process is:

v3.1 + v3.2



Policy Diff Engine



Changed Requirement



Affected Control



Affected Processes



Affected Transactions

For example, a previous ₹4,50,000 transaction that was compliant under v3.1 may become non-compliant under v3.2.

This is much more useful than simply storing the new policy.

15. Policy Diff Engine

The **Policy Diff Engine** is not a pretrained AI model.

It can be implemented using Python and structured/text comparison.

It identifies changes such as:

- Threshold changes

- Added requirements
- Removed requirements
- Changed approval roles
- Changed conditions
- Changed deadlines
- Changed mandatory documents

The LLM can then help interpret complex textual changes.

So the architecture is:

Policy Diff Engine



Exact differences



LLM



Meaning / interpretation



Impact Engine

16. Policy Impact Engine

The Impact Engine determines what is affected by a policy change.

For example:

Policy threshold changed



Control affected



Procurement process affected



Transactions between ₹3L–₹5L identified



Transactions re-evaluated

It can use:

- Python
- MySQL queries
- Structured compliance controls
- Transaction data
- Policy metadata

The existing **Policy Change Impact Agent** can coordinate this process.

17. Automatic Compliance Test Generation

Another important technique is automatically generating compliance tests from policy controls.

For example:

Policy:

Finance approval required for purchases > ₹3,00,000

The system can generate:

Test 1:

₹2,50,000 + no approval → PASS

Test 2:

₹3,50,000 + approval → PASS

Test 3:

₹3,50,000 + no approval → FAIL

Test 4:

₹3,00,000 + no approval → Boundary Test

This allows the system to test whether compliance controls are actually working.

18. Compliance Testing

Aegis-AI can have a **Compliance Test Lab** that evaluates controls.

Testing can include:

- Positive test cases
- Negative test cases

- Boundary cases
- Policy-change test cases
- Historical transaction tests
- Control effectiveness tests

Example:

Procurement Control

Transactions Tested: 1,250

Passed: 1,174

Failed: 58

Review: 18

These numbers would be generated from actual test data rather than being fixed values.

19. Policy-to-Control Compiler

Another important research feature is the **Policy-to-Control Compiler**.

It converts:

Natural-language policy



Structured control

For example:

"Purchases above ₹3 lakh require Finance approval."



Control:

`purchase_amount > 300000`

Required Role:

Finance

Action:

`approval_required`

This allows natural-language organizational policies to become executable compliance controls.

20. Policy Conflict Detection

Aegis-AI can also identify conflicts between policies.

Example:

Procurement Policy:

Approval required above ₹3 lakh

Finance Policy:

Approval required above ₹5 lakh

The system can detect the inconsistency and flag it for human review instead of silently choosing one requirement.

21. Policy Ambiguity Detection

The system can identify unclear policy statements.

For example:

"Large purchases require additional approval."

The term "**large**" is ambiguous.

Aegis-AI can flag:

Ambiguous requirement detected.

Missing information:

Purchase threshold

Human clarification required.

This prevents the system from inventing a threshold.

22. Compliance Drift Detection

Aegis-AI can monitor compliance rates over time.

For example:

Month 1 → 96% compliant

Month 2 → 94%

Month 3 → 89%

Month 4 → 82%

The system can identify a potential **compliance drift** and alert the compliance team.

23. Risk-Adaptive Agent Permissions

Aegis-AI should not give every AI agent unlimited authority.

Instead:

Low Risk



Automatic action allowed

Medium Risk



Human approval required

High/Critical Risk



Mandatory human intervention

This creates **risk-adaptive autonomy** and makes the system safer for enterprise use.

24. Human-in-the-Loop

Human oversight is particularly important for consequential compliance actions.

Humans can approve:

- Policy publication
- High-risk remediation
- Critical violations
- Ambiguous policies
- Policy conflicts
- Sensitive transactions

This gives Aegis-AI a balance between **automation and human control**.

25. Explainable Compliance Decisions

Every compliance decision should ideally contain:

Decision

+

Applicable Policy

+

Policy Version

+

Relevant Policy Evidence

+

Compliance Rule

+

Observed Transaction Data

+

Risk Level

+

Recommended Action

Therefore, instead of simply saying:

"Transaction is non-compliant."

the system can explain:

Transaction TXN-1003 is non-compliant because the purchase amount exceeds the Finance approval threshold defined in Procurement Policy v3.2, while the required approval is missing.

This makes decisions more **auditable and explainable**.

26. Audit Evidence

Aegis-AI maintains an evidence trail for compliance cases.

Example:

Transaction

↓

Policy Version



Applicable Control



Retrieved Policy Evidence



Rule Evaluation



Risk Assessment



Agent Actions



Remediation



Verification

This provides traceability for audits.

27. Policy What-If Simulation

A possible advanced feature is a **Policy What-If Simulator**.

Example:

Current Policy:

Approval > ₹5 lakh

Proposed Policy:

Approval > ₹3 lakh

The system can simulate:

- Number of affected transactions
- New violations
- Affected controls
- Risk impact
- Expected compliance workload
- Remediation workload

The organization can therefore evaluate a policy change **before applying it**.

28. Compliance Digital Twin

A more advanced extension is a **Compliance Digital Twin** representing:

Policies



Controls



Processes



Departments



Transactions



Risks



Agents

It can be used for simulation and impact analysis.

This is optional, but it provides a strong future research direction.

29. Technology Stack

Frontend

- **React**
- **Vite**
- **Tailwind CSS**
- **React Router**
- **Axios**
- **Recharts**

Backend

- **Python**
- **FastAPI**

- **SQLAlchemy**
- **PyMySQL**
- **Pydantic**

AI / NLP

- **Large Language Model**
- **Groq** as an inference provider
- **Llama or other suitable LLM**
- **RAG**
- **Sentence Transformers**
- **all-MiniLM-L6-v2**
- **LangGraph**

Vector Database

- **ChromaDB**

Relational Database

- **MySQL**

Core Intelligence

- **Deterministic Rule Engine**
- **Policy-to-Control Compiler**
- **Policy Diff Engine**
- **Policy Impact Engine**
- **Compliance Test Generator**
- **Risk Assessment Engine**
- **Policy Conflict Detector**
- **Policy Ambiguity Detector**
- **Compliance Drift Detector**

Development

- **VS Code**
- **Git**
- **GitHub**

30. Testing Technologies

Aegis-AI can use:

API testing

- pytest
- httpx

Functional testing

- pytest

Compliance testing

- Python
- pytest

RAG evaluation

- Ragas

LLM/agent evaluation

- LangSmith

Security testing

- OWASP ZAP
- pytest

End-to-end testing

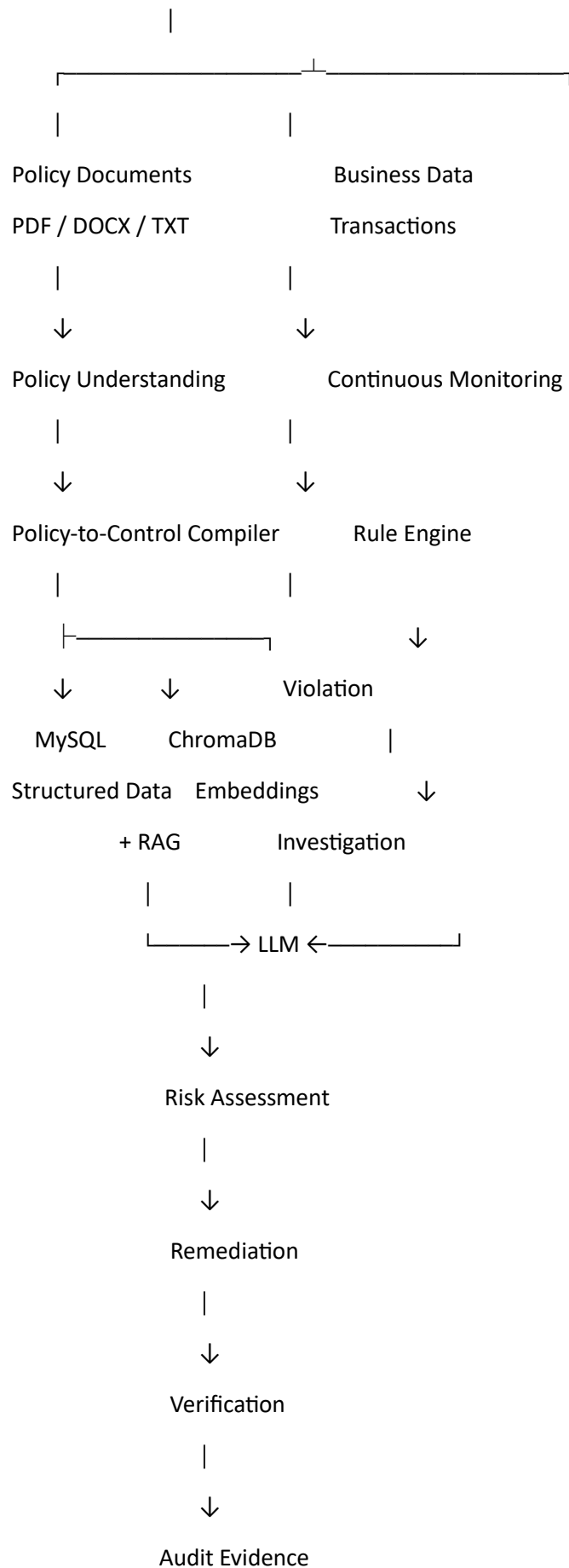
- pytest
- httpx

Security tests can cover:

- Authentication
- Authorization
- SQL injection
- Prompt injection
- Sensitive-data leakage
- Excessive agent permissions
- Malicious policy documents
- Unauthorized remediation

31. Complete Technology Flow

AEGIS-AI



And policy updates create an additional loop:

Old Policy

+

New Policy

↓

Policy Diff

↓

Impact Analysis

↓

Affected Controls

↓

Affected Transactions

↓

Compliance Test Generation

↓

Test Execution

↓

New Violations

↓

Investigation → Remediation → Verification

32. Strongest Novelty of Aegis-AI

The strongest research contribution is **not simply using an LLM or multiple agents**.

The stronger idea is the combination:

**Policy change → automatic difference detection → impact analysis → compliance test generation
→ affected transaction identification → risk assessment → investigation → controlled remediation
→ independent verification → audit evidence**

This creates a **closed-loop adaptive compliance mechanism**.

In simple words:

Aegis-AI does not just understand a policy. It can understand what changed, determine what is affected, test the new requirement, identify newly non-compliant transactions, take controlled corrective action, and verify the result.

33. Main Research Contributions

Your paper can present the following contributions:

1. **Natural-language policies are transformed into structured executable compliance controls.**
2. **RAG provides policy-grounded evidence for AI reasoning.**
3. **Deterministic rules provide reliable exact compliance checking.**
4. **Multi-agent workflow supports investigation, risk assessment, remediation and verification.**
5. **Policy changes are automatically analyzed for their effect on controls and transactions.**
6. **Compliance tests can be generated automatically from policy requirements.**
7. **Risk-adaptive autonomy limits agent actions according to violation severity.**
8. **Compliance decisions maintain traceable policy and evidence information.**
9. **The system supports continuous rather than periodic compliance monitoring.**

34. One-Sentence Description

Aegis-AI is an adaptive multi-agent compliance platform that converts organizational policies into executable controls, continuously monitors business processes using deterministic rules and RAG-based reasoning, analyzes policy changes and their impacts, automatically tests compliance, and performs risk-aware investigation, remediation, and verification with auditable evidence.

This is the **complete technical picture** of your Aegis-AI project and can serve as the base for your **IEEE paper, architecture, methodology, implementation, testing, and presentation.**